
LAPORAN PRATIKUM 1

TEKNOLOGI IOT 2



Oleh :

Nama : Virnarokaren

NPM : 25782115

Kelas : TRI (3D)

Dosen Pengampu

Dr. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, SPd., M.Kom.

Andi Saptono, S.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

LAPORAN PRAKTIKUM 1

FONDASI ARSITEKTUR INTERNET OF THINGS (IoT) DAN MIKROKONTROLER

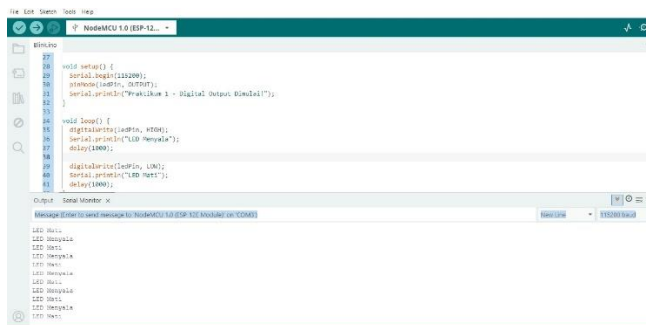
Praktikum 1: Digital Output (Menyalakan LED)

Tujuan

Praktikum ini dilakukan untuk memahami cara mengatur pin digital sebagai output pada mikrokontroler. Pada praktikum ini digunakan LED sebagai komponen output yang dikendalikan melalui program.

Hasil Praktikum

Pada praktikum pertama, LED dihubungkan dengan pin GPIO 5 dan resistor 220 Ohm. Setelah program dijalankan, LED dapat menyala dan mati secara bergantian. Kondisi tersebut terjadi karena program memberikan nilai HIGH dan LOW pada pin secara bergantian dengan jeda waktu satu detik.



```
1 // Praktikum 1 - Digital Output (Menyalakan LED)
2
3 #include <Arduino.h>
4
5 // Definisi pin yang digunakan
6 const int LED_PIN = 5;
7
8 // Fungsi untuk menyalakan LED
9 void digitalWrite_LED_High() {
10   digitalWrite(LED_PIN, HIGH);
11 }
12
13 // Fungsi untuk mematikan LED
14 void digitalWrite_LED_Low() {
15   digitalWrite(LED_PIN, LOW);
16 }
17
18 // Fungsi untuk menyalakan LED selama 1 detik
19 void digitalWrite_LED_High_1s() {
20   digitalWrite_LED_High();
21   delay(1000);
22 }
23
24 // Fungsi untuk mematikan LED selama 1 detik
25 void digitalWrite_LED_Low_1s() {
26   digitalWrite_LED_Low();
27   delay(1000);
28 }
29
30 // Fungsi utama
31 void setup() {
32   pinMode(LED_PIN, OUTPUT);
33   Serial.begin(9600);
34   Serial.println("Praktikum 1 - Digital Output (Menyalakan LED)");
35 }
36
37 void loop() {
38   digitalWrite_LED_High_1s();
39   digitalWrite_LED_Low_1s();
40 }
```



Pembahasan

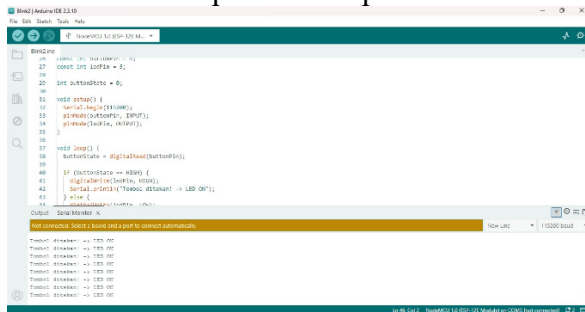
Berdasarkan hasil simulasi, rangkaian dapat berjalan sesuai dengan program yang dibuat. Ketika pin GPIO 5 diberikan kondisi HIGH, LED menyala. Setelah menunggu selama satu detik, program mengubah kondisi pin menjadi LOW sehingga LED mati. Proses tersebut dilakukan terus menerus karena perintah berada di dalam fungsi loop().

Penggunaan resistor pada rangkaian berfungsi untuk membatasi arus yang mengalir menuju LED. Dari percobaan ini dapat dipahami bahwa pin digital dapat digunakan untuk mengendalikan perangkat output dengan memberikan kondisi HIGH atau LOW.

Praktikum 2: Digital Input (Membaca Push Button)

Tujuan

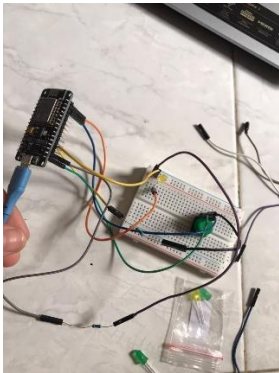
Praktikum kedua bertujuan untuk mengetahui cara membaca input digital menggunakan push button. Tombol digunakan sebagai input, sedangkan LED digunakan sebagai output yang memberikan respon terhadap kondisi tombol.



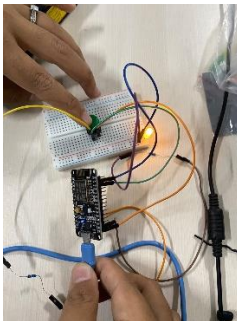
Hasil Praktikum

Pada percobaan ini, push button dihubungkan dengan GPIO 4 menggunakan rangkaian *pull-down*. Ketika tombol tidak ditekan, input yang dibaca berada pada kondisi LOW. Saat tombol ditekan, tegangan 3.3V masuk ke pin sehingga input berubah menjadi HIGH dan LED menyala.

ketika tombol tidak ditekan



ketika tombol ditekan



Pembahasan

Dari hasil simulasi dapat dilihat bahwa mikrokontroler mampu membaca kondisi push button dengan baik. Program membaca keadaan tombol menggunakan `digitalRead()`. Jika hasil pembaca

adalah HIGH, program akan menyalakan LED. Sebaliknya, jika hasil pembacaan LOW, LED akan dimatikan.

Rangkaian *pull-down* membuat pin input berada pada kondisi LOW ketika tombol tidak ditekan. Saat tombol ditekan, pin mendapatkan tegangan dari 3.3V sehingga berubah menjadi HIGH. Dengan percobaan ini dapat diketahui hubungan antara input digital dan output digital pada mikrokontroler.

HASIL JAWABAN LATIHAN

Latihan 1 – Pemahaman Logika Pin

Soal: Jelaskan apa yang terjadi jika tombol *Active-High (Pull-Down)* dipasang pada D8 (GPIO 15) dan ditekan ketika NodeMCU baru diberi daya.

Jawaban

Jika tombol dipasang pada D8 atau GPIO 15 dan tombol ditekan tepat ketika ESP32 baru dinyalakan, proses awal ESP32 dapat terganggu. GPIO 15 mempunyai fungsi khusus pada saat proses *booting*, sehingga kondisinya harus sesuai dengan ketentuan agar mikrokontroler dapat masuk ke proses *booting* normal.

Pada rangkaian *pull-down*, ketika tombol ditekan maka GPIO 15 akan mendapatkan tegangan 3.3V atau HIGH. Kondisi tersebut tidak sesuai dengan kondisi yang dibutuhkan GPIO 15 ketika proses *booting*, sehingga ESP32 dapat mengalami kegagalan *booting* atau tidak berjalan sebagaimana mestinya.

Kesimpulan:

GPIO 15 sebaiknya tidak digunakan dengan konfigurasi seperti tersebut karena perubahan kondisinya saat ESP32 mulai menyala dapat memengaruhi proses *booting*.

Latihan 2 – Implementasi Kode

Soal: Modifikasi program Praktikum 1 agar LED menyala selama 200 ms dan mati selama 800 ms.

Jawaban

Pada program awal, LED memiliki waktu menyala dan mati masing-masing 1000 ms. Agar waktu keduanya berbeda, nilai `delay()` diubah menjadi 200 ms untuk kondisi LED menyala dan 800 ms untuk kondisi LED mati.

Bagian program yang diubah Sehingga bagian `loop()` menjadi:



The screenshot shows the Arduino IDE interface. The main editor window displays the code for `Blink1.ino`. The code includes a comment linking to the Arduino Blink example, a setup function that initializes the LED pin and serial communication, and a loop function that toggles the LED state with specific delays. The Serial Monitor at the bottom shows the output of the program, which is "LED Menyala" followed by "LED Mati" in a repeating pattern.

```
21
22  https://docs.arduino.cc/built-in-examples/basics/Blink/
23  */
24
25  // the setup function runs once when you press reset or power the board
26  const int ledPin = 5;
27
28  void setup() {
29    Serial.begin(115200);
30    pinMode(ledPin, OUTPUT);
31    Serial.println("Praktikum 1 - Digital Output Dimulai!");
32  }
33
34  void loop() {
35    digitalWrite(ledPin, HIGH);
36    Serial.println("LED Menyala");
37    delay(200);
38
39    digitalWrite(ledPin, LOW);
40    Serial.println("LED Mati");
41    delay(800);
42  }
```

Output Serial Monitor X

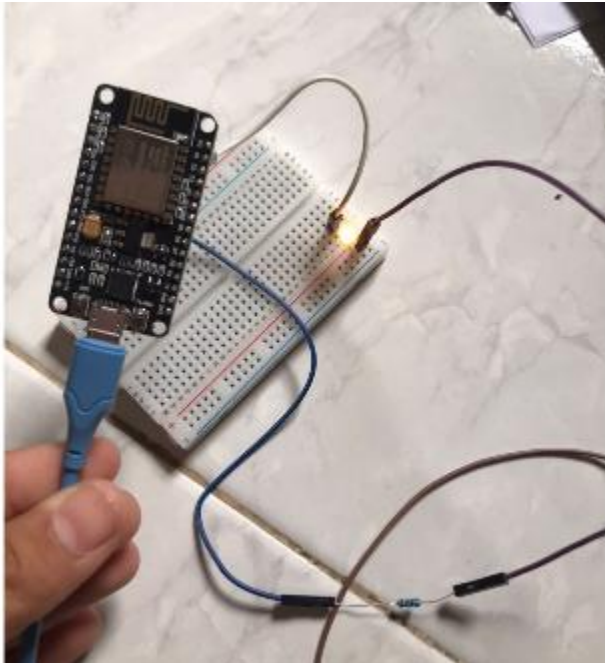
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

LED Menyala
LED Mati
LED Menyala
LED Mati

Ln 41, Col 12 NodeMCU 1.0 (ESP-12E Module) on COM3 2

Hasil

Setelah program dijalankan, LED terlihat menyala dalam waktu yang lebih singkat dan kemudian mati lebih lama. Pola tersebut terus berulang selama program berjalan.



Latihan 3 – Analisis Sirkuit

Soal: Apa yang terjadi jika kaki LED dipasang terbalik, yaitu Anoda ke GND dan Katoda ke D1? Apakah program mengalami error kompilasi?

```
Blink1.ino
21
22   https://docs.arduino.cc/built-in-examples/basics/Blink/
23   */
24
25   // the setup function runs once when you press reset or power the board
26   const int ledPin = 5;
27
28   void setup() {
29     Serial.begin(115200);
30     pinMode(ledPin, OUTPUT);
31     Serial.println("Praktikum 1 - Digital Output Dimulai!");
32   }
33
34   void loop() {
35     digitalWrite(ledPin, HIGH);
36     Serial.println("LED Menyala");
37     delay(200);
38
39     digitalWrite(ledPin, LOW);
40     Serial.println("LED Mati");
41     delay(800);
42   }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

LED Menyala
LED Mati
LED Menyala
LED Mati

New Line 115200 baud

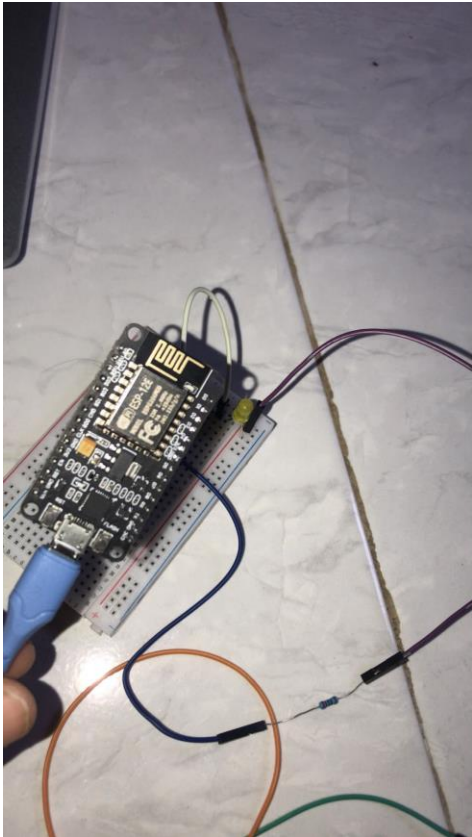
Jawaban

Jika LED dipasang dengan posisi terbalik, yaitu Anoda dihubungkan ke GND dan Katoda dihubungkan ke D1, LED tidak dapat menyala seperti pada rangkaian normal. Hal ini disebabkan oleh arah pemasangan LED yang tidak sesuai sehingga arus tidak mengalir melalui LED dalam arah yang benar.

Walaupun rangkaiannya salah, program tidak mengalami error ketika dikompilasi. Hal tersebut karena program yang dibuat tidak memiliki kesalahan sintaks. Masalahnya terdapat pada pemasangan komponen pada rangkaian. LED merupakan komponen yang memiliki polaritas sehingga pemasangannya harus diperhatikan.

Hasil Pengamatan

Pada simulasi, program tetap dapat dijalankan, tetapi LED tidak memberikan cahaya seperti ketika dipasang dengan polaritas yang benar.



Latihan 4 – Pengembangan Algoritma

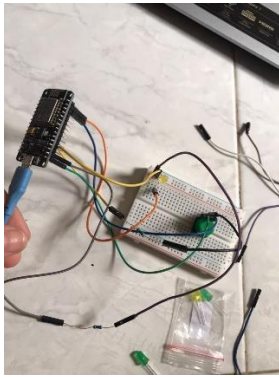
Soal: Ubah kondisi program sehingga ketika tombol tidak ditekan LED menyala, sedangkan ketika tombol ditekan LED mati.

Jawaban

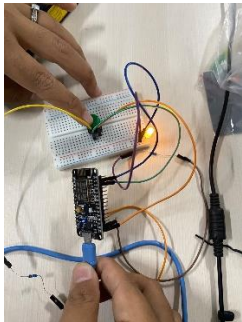
Pada latihan ini kondisi if-else diubah sehingga kerja LED menjadi kebalikan dari program Praktikum 2. Ketika tombol tidak ditekan, input bernilai LOW sehingga LED dinyalakan. Ketika tombol ditekan, input menjadi HIGH dan LED dimatikan.

Dengan perubahan tersebut, LED akan berada dalam keadaan menyala ketika tombol dilepas. Ketika tombol ditekan, LED akan langsung berubah menjadi mati.

Saat tidak di tekan



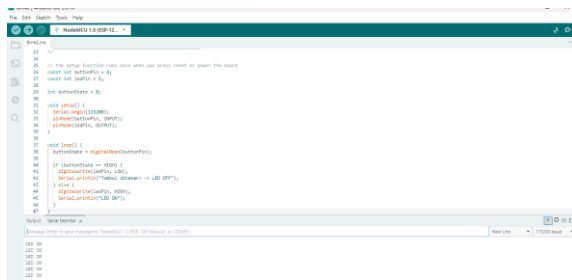
Saat tombol di tekan



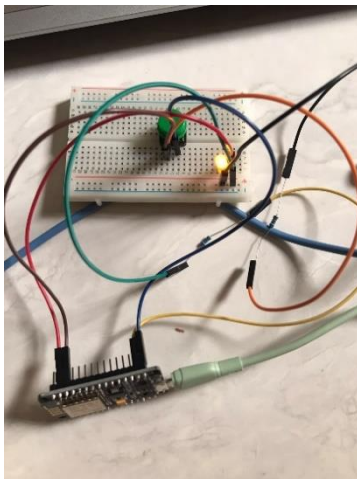
TUGAS – SISTEM SAKELAR TOGGLE (LATCHING)

Tujuan

Tugas ini bertujuan untuk membuat tombol yang dapat bekerja seperti sakelar. Jadi, LED tidak hanya menyala ketika tombol ditekan, tetapi kondisi LED akan tetap tersimpan sampai tombol ditekan kembali.



```
1 // The setup function runs once when you press reset or power the board
2
3 const int buttonPin = 4;
4 const int ledPin = 13;
5 int ledState = 0;
6
7 void setup() {
8   pinMode(buttonPin, INPUT);
9   pinMode(ledPin, OUTPUT);
10 }
11
12 void loop() {
13   if (digitalRead(buttonPin) == HIGH) {
14     digitalWrite(ledPin, !ledState);
15     Serial.println("Tombol ditekan! -> LED: " + String(ledState));
16     digitalWrite(ledPin, ledState);
17   }
18 }
```



Pembahasan

Pada tugas ini digunakan variabel `ledState` untuk menyimpan kondisi LED. Nilai awal variabel tersebut adalah `false`, sehingga ketika program pertama kali dijalankan LED dalam keadaan mati

Ketika tombol ditekan, program akan mengubah nilai `ledState` menggunakan perintah `ledState = !ledState`. Jika sebelumnya bernilai `false`, nilainya berubah menjadi `true` dan LED menyala. Jika tombol ditekan kembali, nilai tersebut berubah dari `true` menjadi `false` sehingga LED mati.

Pada program juga digunakan `delay(200)` sebagai *debounce*. Bagian ini digunakan untuk mengurangi kemungkinan satu tekanan tombol terbaca lebih dari satu kali akibat adanya pantulan mekanis pada tombol. Program kemudian menunggu sampai tombol dilepas sebelum menerima tekanan berikutnya.

Hasil Pengujian

Berdasarkan hasil pengujian, kondisi LED berubah setiap kali tombol ditekan dan dilepas.

Percobaan	Kondisi Tombol	Kondisi LED
Awal	Belum ditekan	Mati
Tekan pertama	Ditekan lalu dilepas	Menyala
Tekan kedua	Ditekan lalu dilepas	Mati
Tekan ketiga	Ditekan lalu dilepas	Menyala

Dengan demikian, rangkaian sudah bekerja seperti sakelar *toggle*. LED tidak langsung mati ketika tombol dilepas, tetapi tetap mempertahankan kondisi terakhirnya sampai tombol ditekan kembali.

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat dipahami bahwa mikrokontroler dapat digunakan untuk membaca input digital dan mengendalikan output digital. Pada Praktikum 1, pin digital digunakan untuk mengatur LED agar dapat menyala dan mati secara bergantian. Pada Praktikum 2, push button digunakan sebagai input untuk mengontrol kondisi LED.

Dari latihan yang dilakukan, dapat diketahui bahwa pemilihan pin pada mikrokontroler perlu diperhatikan karena beberapa pin mempunyai fungsi khusus pada saat proses *booting*. Selain itu, perubahan nilai `delay()` dapat digunakan untuk mengatur lama waktu LED menyala dan mati.

Pada tugas terakhir, tombol berhasil dikembangkan menjadi sistem *toggle* dengan menggunakan variabel untuk menyimpan kondisi LED. Dengan cara tersebut, satu kali tekanan dapat mengubah kondisi LED dan kondisi tersebut tetap bertahan sampai tombol ditekan kembali. Praktikum ini membantu memahami dasar penggunaan input, output, kondisi if-else, variabel *state*, dan *debouncing* pada mikrokontroler.